

BuloScan

Detector de Noticias Falsas en Tiempo Real

PROGRAMACIÓN DE INTELIGENCIA ARTIFICIAL

Autor: Luis Liébanas Ávila

1. ¿Qué es Buloscan?

Buloscan es una aplicación web que analiza noticias y determina si son fiables, sospechosas o falsas. El usuario puede pegar el texto de cualquier noticia para obtener un veredicto inmediato, o buscar noticias reales por tema a través de NewsAPI.

Cada análisis queda registrado en una cadena de bloques (blockchain) que garantiza que el historial no puede ser manipulado.

Tecnologías usadas

Tecnología	Para qué se usa
FastAPI (Python)	Backend y API REST. Gestiona todas las peticiones del frontend.
HTML + CSS + JS	Frontend. Interfaz web completa en un único fichero index.html.
TextBlob	Análisis de sentimiento: mide si el texto es subjetivo o emotivo.
SQLite	Base de datos local donde se guardan todos los análisis.
Blockchain propia	Registro inmutable de cada análisis. Detecta manipulaciones.
NewsAPI	Fuente de noticias reales para búsquedas por tema.

Estructura del proyecto

Fichero	Qué hace
main.py	Punto de entrada. Define los endpoints de la API y arranca el servidor.
index.html	Frontend completo: formulario de análisis, buscador e historial.
core/analyzer.py	Motor de análisis: calcula el score de falsedad con 3 factores.
core/blockchain.py	Implementa la cadena de bloques con Proof of Work.
core/database.py	Guarda y consulta los análisis en SQLite.
news/news_fetcher.py	Llama a NewsAPI y normaliza los artículos obtenidos.
news/news_monitor.py	Gestiona los sensores de noticias por temática.

2. Cómo funciona el análisis

Cada noticia se puntúa con tres factores independientes. El resultado final (score_fake) va de 0.0 (muy fiable) a 1.0 (muy falsa) y se calcula con esta fórmula:

$$\text{score_fake} = \text{fuente} \times 0,30 + \text{lingüístico} \times 0,50 + \text{sentimiento} \times 0,20$$

Factor 1 — Análisis de fuente (30%)

Se comprueba el dominio de la noticia contra dos listas:

- Fuentes fiables conocidas: elpais.com, bbc.com, reuters.com, rtve.es... → score 0.10
- Dominios sospechosos: noticiasfalsas, elmundohoy, okdiario... → score 0.90
- Fuente desconocida o vacía → score 0.55 – 0.72

Factor 2 — Análisis lingüístico (50%)

Es el factor con más peso. Busca en el texto:

- Palabras de alarmismo: urgente, increíble, impactante, catastrófico...
- Frases conspirativas: 'el gobierno oculta', 'lo que los medios no cuentan'...
- Llamadas a compartir: 'comparte antes de que lo borren', 'difunde esto'...
- Afirmaciones imposibles: 'tierra plana', 'chips en las vacunas', 'ha resucitado'...
- Patrones de escritura: MAYÚSCULAS EXCESIVAS, !!, ??, puntos suspensivos...

También bonifica el lenguaje periodístico riguroso: atribución de fuentes, datos precisos, lenguaje formal.

Factor 3 — Análisis de sentimiento (20%)

Usa TextBlob para medir dos cosas del texto:

- Subjetividad (0 = objetivo, 1 = muy subjetivo): las noticias falsas tienden a ser muy subjetivas.
- Polaridad (-1 = muy negativo, +1 = muy positivo): la polaridad extrema es señal de alerta.

Veredicto final

Score	Veredicto	Significado
< 0.38	✓ FIABLE	Sin indicadores significativos de falsedad.
0.38 – 0.58	⚠ SOSPECHOSA	Algunos indicadores dudosos. Verificar antes de compartir.
> 0.58	✗ FALSA	Múltiples indicadores de falsedad detectados.

3. Código importante

3.1 Diccionarios del analizador (core/analyzer.py)

El analizador usa cuatro listas que definen su 'conocimiento'. Aquí se definen las palabras sospechosas, las creíbles, los patrones de escritura y los dominios malos:

```
# Palabras y frases que aparecen frecuentemente en noticias falsas o sensacionalistas
PALABRAS_SOSPECHOSAS = [
    # Alarmismo y urgencia falsa
    "urgente", "exclusivo", "increíble", "impactante", "sorprendente",
    "shockeante", "brutal", "devastador", "catastrófico", "apocalíptico",
    # Conspiraciones y ocultamiento
    "lo que los medios ocultan", "te sorprenderá", "nadie te cuenta",
    "el gobierno oculta", "censurado", "prohibido", "lo que no quieren que sepas",
    "la verdad que esconden", "conspiracion", "conspiración",
    # ... (otras palabras sospechosas)
]
```

```
# Palabras que sugieren rigor periodístico y credibilidad
PALABRAS_CREDIBLES = [
    # Fuentes verificables
    "según el ministerio", "declaró el portavoz", "confirmó el gobierno",
    "informó reuters", "según datos oficiales", "fuentes oficiales",
    "el informe señala", "el estudio concluye", "investigadores de",
    # Lenguaje formal y preciso
    "por ciento", "millones de euros", "en declaraciones a",
    "en rueda de prensa", "según el instituto", "datos del ine",
    "el tribunal", "el parlamento", "la comisión europea",
    # Atribución clara
    "ha declarado", "ha confirmado", "ha anunciado", "ha publicado"
]

# Patrones de escritura asociados a desinformación
PATRONES_SOSPECHOSOS = [
    r"[A-ZÁÉÍÓÚ\s]{6,}",      # Texto en MAYÚSCULAS excesivas (6+ chars)
    r"!\{2,}",                  # Múltiples signos de exclamación !!
    r"\?\{2,}",                  # Múltiples signos de interrogación ??
    r"\.\{3,}",                  # Puntos suspensivos excesivos ...

```

3.2 Score ponderado (core/analyzer.py)

Aquí es donde se combinan los tres factores para obtener la puntuación final. El lingüístico tiene más peso (50%) porque es el más fiable para detectar fake news:

```
score_fake = (  
    score_fuente * 0.30 +  
    score_linguistico * 0.50 +  
    score_sentimiento * 0.20  
)  
score_fake = round(min(max(score_fake, 0.0), 1.0), 4) # Clamping 0-1  
  
# Veredicto según umbrales  
if score_fake < self.UMBRAL_FIABLE:  
    veredicto = "FIABLE"  
elif score_fake < self.UMBRAL_SOSPECHOSA:  
    veredicto = "SOSPECHOSA"  
else:  
    veredicto = "FALSA"
```

Justo después, se asigna el veredicto comparando el score con los umbrales 0.38 y 0.58.

3.3 Hash del bloque (core/blockchain.py)

Cada bloque se identifica con un hash SHA-256 único. Si alguien modifica cualquier dato del bloque, el hash cambia y la cadena deja de ser válida:

```
7 class Block:
8     def __init__(self, index, analysis_data, previous_hash):
9
10         self.timestamp = time.time()
11         self.analysis_data = analysis_data # Datos del análisis de la noticia
12         self.previous_hash = previous_hash
13         self.nonce = 0
14         self.hash = self.compute_hash()
15
16     def compute_hash(self):
17         block_string = json.dumps({
18             "index": self.index,
19             "timestamp": self.timestamp,
20             "analysis_data": self.analysis_data,
21             "previous_hash": self.previous_hash,
22             "nonce": self.nonce
23         }, sort_keys=True)
24         return hashlib.sha256(block_string.encode()).hexdigest()
25
26     def to_dict(self):
27         # Convierte el bloque a diccionario para guardarlo en SQLite o devolverlo como JSON.
28         return {
29             "index": self.index,
30             "timestamp": self.timestamp,
31             "analysis_data": self.analysis_data,
32             "previous_hash": self.previous_hash,
33             "nonce": self.nonce,
34             "hash": self.hash
```

3.4 Proof of Work (core/blockchain.py)

Antes de añadir un bloque a la cadena hay que 'minarlo': buscar un número (nonce) que haga que el hash empiece por dos ceros. Esto hace que alterar un bloque antiguo sea computacionalmente costoso:

```
        return mined_block
    def proof_of_work(self, block):
        while not block.hash.startswith("0" * self.DIFFICULTY):
            block.nonce += 1
            block.hash = block.compute_hash()
        return block
    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]
            # El hash almacenado debe coincidir con el recalculado
            if current.hash != current.compute_hash():
                return False
            # El enlace con el bloque anterior debe ser correcto
            if current.previous_hash != previous.hash:
                return False
        return True
```

3.5 Endpoint principal (main.py)

Cuando el usuario pulsa Analizar, el frontend llama a este endpoint. Recibe el texto, construye el objeto noticia, lo analiza y guarda el resultado en SQLite y en la blockchain:

```
@app.post("/analizar/texto")
def analizar_texto(request: TextoRequest):
    if not request.texto.strip():
        raise HTTPException(status_code=400, detail="El texto no puede estar vacío")

    # Construimos una noticia artificial con el texto del usuario
    noticia = {
        "titulo": request.texto[:150], # Usamos los primeros 150 chars como título
        "descripcion": request.texto,
        "url": request.url or f"texto-manual-{hash(request.texto)}",
        "fuente": request.fuente,
        "dominio": _extraer_dominio(request.url) if request.url else "",
        "autor": "Introducido manualmente",
        "publicada_en": "",
        "imagen_url": "",
        "es_fiable": False
    }

    resultado = analizar_noticia(noticia)
    _registrar_resultado(resultado)
    return resultado
```

4. La aplicación en funcionamiento

Pantalla completa con un análisis completado. Se puede ver el veredicto en rojo, la barra de score y el desglose por los tres factores:

01 - ANALIZAR NOTICIA

URGENTE!! la Francisco Franco está vivo en 2026, fue ocultada por su familia.

Fuente [opcional, ej: elpais.com]

URL de la noticia [opcional]

🔍 Analizar

✖ FALSA

score: 72%

Fuente

Lingüístico

Sentimiento

Score final

Fuente no identificada

Afirmación imposible detectada: 'está vivo'

Texto objetivo y neutral - señal positiva

71.6% probabilidad de ser falsa

5. Cómo instalarlo y usarlo

Instalación

1. Descargar o clonar el repositorio.
2. Abrir una terminal en la carpeta del proyecto.
3. Instalar dependencias: `pip install -r requirements.txt`
4. Arrancar el servidor: `python -m uvicorn main:app --reload`
5. Abrir el navegador en: `http://127.0.0.1:8000`

Uso básico

Analizar una noticia manualmente:

6. Pegar el texto en el campo grande de la izquierda.
7. Indicar la fuente y URL si se tienen (opcional).
8. Pulsar Analizar. En menos de 2 segundos aparece el veredicto con el desglose.

Buscar noticias por tema:

9. Escribir un tema en el campo inferior izquierdo (ej: vacunas, elecciones).
10. Seleccionar idioma ES o EN.
11. Pulsar Buscar y analizar. El sistema obtiene noticias reales de NewsAPI y las analiza.

Endpoints de la API

Método	Endpoint	Descripción
GET	/	Sirve el frontend (index.html)
POST	/analizar/texto	Analiza un texto introducido manualmente
POST	/analizar/busqueda	Busca y analiza noticias por tema en NewsAPI
GET	/noticias	Historial de noticias analizadas
GET	/estadisticas	Contadores globales del sistema
GET	/blockchain/validar	Verifica la integridad de la cadena de bloques